Pointers in C,C++

Outline

- Computer Memory Structure
- Addressing Concept
- Introduction to Pointer
- Pointer Manipulation
- Summary

Computer Memory Revisited

- Computers store data in memory slots
- Each slot has an *unique address*
- Variables store their values like this:

Addr	**Content**	**Addr**	**Content**	**Addr**	**Content**	**Addr**	**Content**
1000	i: 37	1001	j: 46	1002	k: 58	1003	m: 74
1004	a[0]: ‘a’	1005	a[1]: ‘b’	1006	a[2]: ‘c’	1007	a[3]: ‘\0’
1008	ptr: 1001	1009	…	1010		1011	

Computer Memory Revisited

- Altering the value of a variable is indeed changing the content of the memory
  - e.g. i = 40;

Addr	Content	Addr	Content	Addr	Content	Addr	Content
1000	i: 40	1001	j: 46	1002	k: 58	1003	m: 74
1004	a[0]: 'a'	1005	a[1]: 'b'	1006	a[2]: 'z'	1007	a[3]: '\0'
1008	ptr: 1001	1009	…	1010		1011	

Addressing Concept



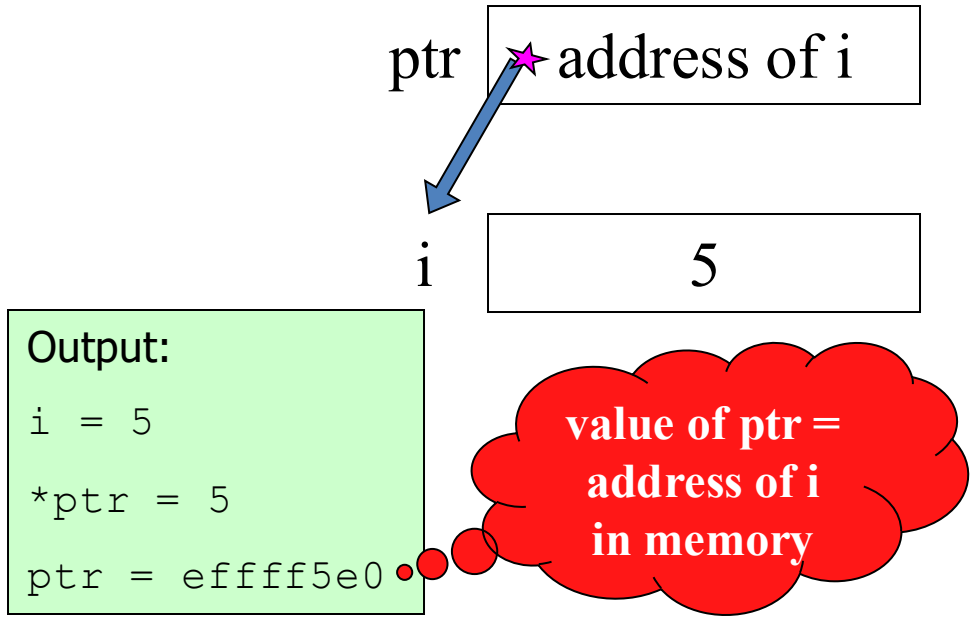
- Pointer stores the **address** of another entity
- It **refers** to a memory location

```
int i = 5;
int* ptr;                          /* declare a pointer variable */
ptr = &i;                          /* store address-of i to ptr */
*ptr = 10;    /* refer to contents a that ptr address*/
```

What actually *ptr* is?

- **ptr** is a variable storing **an address**
- ptr is **NOT** storing the actual value of i

```
int i = 5;
int* ptr;
ptr = &i;
printf("i = %d\n", i);
printf("*ptr = %d\n", *ptr);
printf("ptr = %p\n", ptr);
```



Twin Operators

- &: Address-of operator
  - Get the *address* of an entity
    - e.g. `ptr = &j;`

Addr	Content	Addr	Content	Addr	Content	Addr	Content
1000	i: 40	1001	j: 33	1002	k: 58	1003	m: 74
1004	ptr: 1001	1005		1006		1007	

Twin Operators

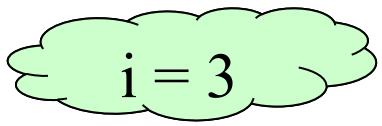
- *: De-reference operator
  - Refer to the *content* of the referee
    - e.g. **`*ptr = 99;`**

Addr	Content	Addr	Content	Addr	Content	Addr	Content
1000	i: 40	1001	j: 99	1002	k: 58	1003	m: 74
1004	ptr: 1001	1005		1006		1007	

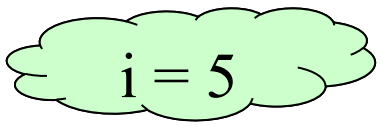
Example: Pass by Pointer

- Modify behaviour in argument passing

```
void f(int j)
{
  j = 5;
}
void g()
{
  int i = 3;
  f(i);
}
```




```
void f(int *ptr)
{
  *ptr = 5;
}
void g()
{
  int i = 3;
  f(&i);
}
```



An Illustration

```
int i = 5, j = 10;
int* ptr;
ptr = &i;
*ptr = 3;
ptr = &j;
*ptr = -2;
```

Data Table			
Name	Type	Description	Value
i	int	integer variable	5
j	int	integer variable	10

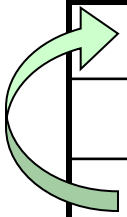
An Illustration

```
int i = 5, j = 10;
int* ptr;     /* declare a pointer-to-integer variable */
ptr = &i;
*ptr = 3;
ptr = &j;
*ptr = -2;
```

Data Table			
Name	Type	Description	Value
i	int	integer variable	5
j	int	integer variable	10
ptr	int *	integer pointer variable	

An Illustration

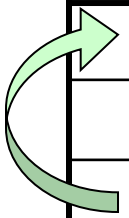
```
int i = 5, j = 10;
int* ptr;
ptr = &i;     /* store address-of i to ptr */
*ptr = 3;
ptr = &j;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	5
j	int	integer variable	10
ptr	int *	integer pointer variable	address of i
*ptr	int	de-reference of ptr	5

An Illustration

```
int i = 5, j = 10;
int* ptr;
ptr = &i;
*ptr = 3;
ptr = &j;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	3
j	int	integer variable	10
ptr	int *	integer pointer variable	address of i
*ptr	int	de-reference of ptr	3

An Illustration

```
int i = 5, j = 10;
int* ptr;
ptr = &i;
*ptr = 3;
ptr = &j;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	3
j	int	integer variable	10
ptr	int *	integer pointer variable	address of j
*ptr	int	de-reference of ptr	10

An Illustration

```
int i = 5, j = 10;
int* ptr;
ptr = &i;
*ptr = 3;
ptr = &j;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	3
j	int	integer variable	-2
ptr	int *	integer pointer variable	address of j
*ptr	int	de-reference of ptr	-2

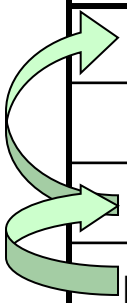
An Illustration

```
int i = 5, j = 10;
int* ptr;
int** pptr;  /* declare a pointer-to-pointer-to-integer variable */
ptr = &i;
pptr = &ptr;
*ptr = 3;
**pptr = 7;
ptr = &j;
**pptr = 9;
*pptr = &i;
*ptr = -2;
```

Data Table			
Name	Type	Description	Value
i	int	integer variable	5
j	int	integer variable	10
ptr	int *	integer pointer variable	
pptr	int **	integer pointer pointer variable	
		Double Indirection	

An Illustration

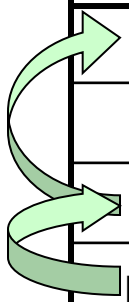
```
int i = 5, j = 10;
int *ptr;
int **pptr;
ptr = &i;
pptr = &ptr; /* store address-of ptr to pptr */
*ptr = 3;
**pptr = 7;
ptr = &j;
**pptr = 9;
*pptr = &i;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	5
j	int	integer variable	10
ptr	int *	integer pointer variable	address of i
pptr	int **	integer pointer pointer variable	address of ptr
*pptr	int *	de-reference of pptr	value of ptr (address of i)

An Illustration

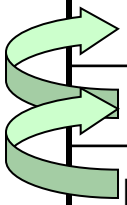
```
int i = 5, j = 10;
int *ptr;
int **pptr;
ptr = &i;
pptr = &ptr;
*ptr = 3;
**pptr = 7;
ptr = &j;
**pptr = 9;
*pptr = &i;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	7
j	int	integer variable	10
ptr	int *	integer pointer variable	address of i
pptr	int **	integer pointer pointer variable	address of ptr
**pptr	int	de-reference of de-reference of pptr	7

An Illustration

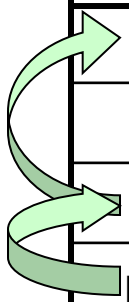
```
int i = 5, j = 10;
int *ptr;
int **pptr;
ptr = &i;
pptr = &ptr;
*ptr = 3;
**pptr = 7;
ptr = &j;
**pptr = 9;
*pptr = &i;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	7
j	int	integer variable	9
ptr	int *	integer pointer variable	address of j
pptr	int **	integer pointer pointer variable	address of ptr
**pptr	int	de-reference of de-reference of pptr	9

An Illustration

```
int i = 5, j = 10;
int *ptr;
int **pptr;
ptr = &i;
pptr = &ptr;
*ptr = 3;
**pptr = 7;
ptr = &j;
**pptr = 9;
*pptr = &i;
*ptr = -2;
```



Data Table			
Name	Type	Description	Value
i	int	integer variable	7
j	int	integer variable	9
ptr	int *	integer pointer variable	address of i
pptr	int **	integer pointer pointer variable	address of ptr
*pptr	int *	de-reference of pptr	value of ptr (address of i)

```
int i = 2, j = 8;

int* ptr;

int** pptr;

ptr = &i;

pptr = &ptr;

*ptr = 7;

**pptr = 12;

ptr = &j;

**pptr = 34;

*pptr = &i;

*ptr = -25;
```

Exercise: Show the values similarly to above for each line of code.

Pointer Arithmetic

- What's `ptr + 1`?

➔The next memory location!

- What's `ptr - 1`?

➔The previous memory location!

- What's `ptr * 2` and `ptr / 2`?

➔Invalid operations!!!

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```

Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	?
a[3]	float	float array element (variable)	?
ptr	float *	float pointer variable	
*ptr	float	de-reference of float pointer variable	?

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	?
a[3]	float	float array element (variable)	?
ptr	float *	float pointer variable	address of a[2]
*ptr	float	de-reference of float pointer variable	?

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	?
ptr	float *	float pointer variable	address of a[2]
*ptr	float	de-reference of float pointer variable	3.14

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	?
ptr	float *	float pointer variable	address of a[3]
*ptr	float	de-reference of float pointer variable	?

Pointer Arithmetic and Array

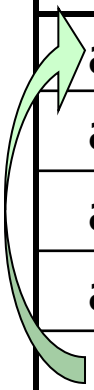
```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	9.0
ptr	float *	float pointer variable	address of a[3]
*ptr	float	de-reference of float pointer variable	9.0

Pointer Arithmetic and Array

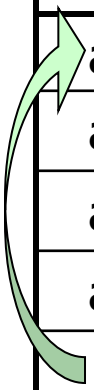
```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	?
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	9.0
ptr	float *	float pointer variable	address of a[0]
*ptr	float	de-reference of float pointer variable	?

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	6.0
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	9.0
ptr	float *	float pointer variable	address of a[0]
*ptr	float	de-reference of float pointer variable	6.0

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	6.0
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	3.14
a[3]	float	float array element (variable)	9.0
ptr	float *	float pointer variable	address of a[2]
*ptr	float	de-reference of float pointer variable	3.14

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```



Data Table			
Name	Type	Description	Value
a[0]	float	float array element (variable)	6.0
a[1]	float	float array element (variable)	?
a[2]	float	float array element (variable)	7.0
a[3]	float	float array element (variable)	9.0
ptr	float *	float pointer variable	address of a[2]
*ptr	float	de-reference of float pointer variable	7.0

Pointer Arithmetic and Array

```
float a[4];
float* ptr;
ptr = &(a[2]);
*ptr = 3.14;
ptr++;
*ptr = 9.0;
ptr = ptr - 3;
*ptr = 6.0;
ptr += 2;
*ptr = 7.0;
```

- Type of `a` is `float *`
- `a[2]` ⬅➡ `*(a + 2)`

  `ptr = &(a[2])`

  ➔`ptr = &(*(a + 2))`

  ➔`ptr = a + 2`
- `a` is a memory address *constant*
- `ptr` is a pointer *variable*

More Pointer Arithmetic

- What if `a` is a `double` array?
- A `double` *may* occupy more memory slots!
  - Given **`double *ptr = a;`**
  - What's **`ptr + 1`** then?

Addr	Content	Addr	Content	Addr	Content	Addr	Content
1000	a[0]: 37.9	1001	…	1002	…	1003	…
1004	a[1]: 1.23	1005	…	1006	…	1007	…
1008	a[2]: 3.14	1009	…	1010	…	1011	…

More Pointer Arithmetic

- Arithmetic operators + and – *auto-adjust* the address offset
- According to the *type* of the pointer:
  - 1000 + sizeof(double) = 1000 + 4 = 1004

Addr	Content	Addr	Content	Addr	Content	Addr	Content
1000	a[0]: 37.9	1001	…	1002	…	1003	…
1004	a[1]: 1.23	1005	…	1006	…	1007	…
1008	a[2]: 3.14	1009	…	1010	…	1011	…

Exercise

- Write code to output the 5 integers in an array using pointer arithmetic.

```
int arr[] = {4,6,7,8,9};
int* ptr= arr;
int len=5;
```

Exercise

- Write a a function called

  int sum(int* ptr,int numValues)

- The function uses pointer arithmetic to return the sum of numValues number of values.

```
int main()
{
    int a[]={4,6,7,8,9};
    cout << sum(a,5);
}
```

Exercise

- Write code that adds the array elements until it reaches a 0.

```
int main()
{
    int a[]={4,6,7,8,9,0};
    int sum=0;
    for(int *ptr=a;;….)
    {
        sum +=*ptr;
    }
```

Pointer to nullptr

```
int *ptr;
//un-initialised pointer could contain any value
..
//set pointer = to nullptr value.
ptr =nullptr;
……
//check if pointer points to anything…
If (ptr== nullptr )
{
}
```

Comparsion with pass by reference and pass by pointer

```
#include <iostream>
 using namespace std;
void swap(int* x, int* y)
 {
int z = *x; *x=*y; *y=z;
}

void swap(int &x, int& y)
 {
int z = x; x=y; y=z;
 }
```

```
int main() {
int a = 45; int b = 35;
cout<<"Before Swap\n";
cout<<"a="<<a<<" b="<<b<<"\n";
swap(&a,&b);
cout<<"After Swap with pass by pointer\n";
 cout<<"a="<<a<<" b="<<b<<"\n";
swap(a,b);
 cout<<"After Swap with pass by reference\n"
cout<<"a="<<a<<" b="<<b<<"\n"; }
```

Output

Before Swap a=45 b=35

After Swap with pass by pointer a=35 b=45

After Swap with pass by reference a=45 b=35

Comparsion with pass by reference and Pass by pointer

- Very Similar
- Pass by reference seen as neater code
- Use pass by reference when not dealing with null values.
- Generally pass by reference parameter should be a const.
- Pass by pointer can pass a null pointer, pass by reference cannot.

Advice and Precaution

- Pros
  - Efficiency
  - Convenience
- Cons
  - Error-prone
  - Difficult to debug

Summary

- A pointer stores the **address** (memory location) of another entity
- Address-of operator (&) **gets the address** of an entity
- De-reference operator (*) **makes a reference** to the referee of a pointer
- Pointer and array
- Pointer arithmetic
- Pass by reference and pass by pointer
- Use nullptr pointer